

# Chapitre 1

---

## ✦ Chapitre 1 – Introduction à l'IA générative

Cours : Usage et éthique de l'IA générative – Ingénierie des données, EST Kelaa des Sraghna

Prof. Oussama Lachihab – Septembre 2025

---

### 1) Définition de l'IA générative

- L'**IA générative** est une branche de l'intelligence artificielle capable de **produire du contenu** : texte, images, audio, vidéos, et données synthétiques.
  - **Différence avec l'IA classique** :
    - **IA classique** → reconnaître, classer, prédire (ex. identifier un chat sur une photo).
    - **IA générative** → inventer, produire (ex. créer une image de chat qui n'existe pas).
- 

### 2) Historique de l'IA générative

| Année       | Événement                                                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>2013</b> | Word2Vec (Google) : représentation des mots en vecteurs → première percée dans la génération créative.                   |
| <b>2014</b> | Naissance des <b>GANs</b> (Generative Adversarial Networks).                                                             |
| <b>2018</b> | <b>BERT</b> (Google) : modèles basés sur Transformers, compréhension du contexte global d'une phrase.                    |
| <b>2020</b> | <b>GPT-3</b> (OpenAI) : modèles multimodaux à grande échelle → IA capable de raisonner et créer dans différents formats. |

---

### 3) Fonctionnement de l'IA générative

1. **Entrée** : le **prompt** (texte ou instruction donnée à l'IA).
2. **Traitement** : le **modèle** génère une réponse à partir de ce prompt.
3. **Sortie** : le contenu généré (texte, image, vidéo, code...).

**Exemples d'applications :**

- **Texte / conversation** : dialogue entre un chat et un rat.
  - **Images** : création d'une image d'une femme astronaute.
  - **Vidéos** : un chien en costume de super-héros volant dans le ciel.
  - **Code** : génération d'un script PHP pour afficher "bonjour tout le monde".
  - **Édition d'images** : retouche ou modification créative d'images existantes.
- 

#### 4) Types de modèles d'IA générative

- **Perceptrons / Réseaux de neurones** :
    - Un **perceptron** est l'unité de base d'un réseau de neurones artificiels, inspiré des neurones biologiques.
    - Un **réseau de neurones** est un ensemble de perceptrons interconnectés pour traiter et générer des informations.
- 

#### 5) Applications de l'IA générative

- Génération de contenu créatif (texte, images, vidéos).
  - Assistance dans le développement de logiciels (ex. génération de code).
  - Chatbots et conversations intelligentes.
  - Retouche et édition automatique de médias.
- 

#### 6) Risques et limites

- Production de contenus **incorrects ou biaisés**.
  - Problèmes d'**éthique** et de propriété intellectuelle.
  - Dépendance aux **données d'entraînement** et aux **algorithmes** utilisés.
- 

Qsm

Parfait ! Voici quelques **questions (QCM / QSM) sur ce chapitre** avec leurs **réponses corrigées**, basées sur ton cours sur l'IA générative :

---

### 1) Définition et concepts

**Q1** : Quelle est la principale différence entre l'IA classique et l'IA générative ?

- a) L'IA classique invente du contenu, l'IA générative prédit seulement.
- b) L'IA classique reconnaît et classe, l'IA générative produit du contenu. ✓
- c) L'IA classique est plus récente que l'IA générative.
- d) Aucune différence.

**Q2 :** L'IA générative peut produire :

- a) Texte
  - b) Images
  - c) Audio
  - d) Toutes les réponses ci-dessus ✓
- 

## 2) Historique de l'IA générative

**Q3 :** Quel modèle a été introduit par Google en 2013 et a marqué une percée dans la génération créative ?

- a) GPT-3
- b) BERT
- c) Word2Vec ✓
- d) GAN

**Q4 :** Les GANs (Generative Adversarial Networks) sont apparus en :

- a) 2010
- b) 2014 ✓
- c) 2018
- d) 2020

**Q5 :** BERT (2018) est un modèle basé sur :

- a) Réseaux de neurones convolutifs
  - b) Transformers ✓
  - c) GANs
  - d) Perceptrons
- 

## 3) Fonctionnement de l'IA générative

**Q6 :** Que signifie le terme **prompt** dans le contexte de l'IA générative ?

- a) Le modèle utilisé
- b) L'entrée / instruction donnée à l'IA ✓
- c) Le résultat généré
- d) Le code source

**Q7 :** Parmi les exemples suivants, lequel n'est pas typiquement généré par l'IA générative ?

- a) Image d'une astronaute ✓
- b) Vidéo d'un chien super-héros
- c) Script PHP simple
- d) Reconnaissance d'un chat dans une photo

---

#### 4) Modèles et réseaux de neurones

**Q8 :** Un perceptron est :

- a) Un modèle mathématique inspiré du neurone biologique ✓
- b) Un type d'ordinateur
- c) Une application IA
- d) Un langage de programmation

**Q9 :** Un réseau de neurones est :

- a) Un seul perceptron
- b) Un ensemble de perceptrons interconnectés ✓
- c) Un modèle de génération de vidéos uniquement
- d) Une base de données

---

#### 5) Risques et limites

**Q10 :** Parmi les propositions suivantes, quelles sont des limites de l'IA générative ?

- a) Contenus incorrects ou biaisés ✓
- b) Dépendance aux données d'entraînement ✓
- c) Problèmes d'éthique ✓
- d) Elle ne peut pas produire de texte ✓

*(La réponse d) est fausse, car l'IA générative **peut produire du texte.**)*

---

Voici un **résumé clair et structuré** de la partie sur les **types de modèles d'IA générative et les neurones artificiels** :

---

## ✦ Les types de modèles d'IA générative

### 1) Neurones artificiels (Perceptrons)

#### Neurone biologique

- Reçoit des signaux via **les dendrites**.
- Le **noyau** décide de l'activation.
- Les signaux sortent par **l'axone**.
- Les connexions se font via les **synapses**.

## Neurone artificiel (Perceptron)

- **Entrées ( $x_1, x_2, \dots, x_n$ )** : valeurs reçues.
- **Poids ( $w_1, w_2, \dots, w_n$ )** : importance de chaque entrée.
- **Biais ( $b$ )** : ajustement pour faciliter l'activation.
- **Somme pondérée** :  $(z = \sum (w_i * x_i) + b)$
- **Fonction d'activation ( $f$ )** : transforme  $z$  en sortie  $y$ .

## Exemple

- Entrées :  $x_1=2, x_2=3, x_3=1$
- Poids :  $w_1=0.5, w_2=0.2, w_3=10$
- Biais :  $b=2$
- Somme pondérée :  $(z = (20.5) + (30.2) + (1*10) + 2 = 14.1)$
- Activation (sigmoïde) :  $(y = 1/(1+e^{-z})) \rightarrow$  sortie.

## Fonctions d'activation courantes

- **Sigmoïde** :  $(g(z) = 1/(1+e^{-z}))$
- **Tanh** :  $(g(z) = (e^z - e^{-z})/(e^z + e^{-z}))$
- **ReLU** :  $(g(z) = \max(0, z))$

---

## 2) Architecture des réseaux neuronaux

- **Couche d'entrée** : reçoit les données brutes (texte, image, son).
- **Couches cachées** : extraient progressivement les caractéristiques.
- **Couche de sortie** : résultat final (classe, valeur, image générée).
- **Propagation avant (Forward pass)** : données  $\rightarrow$  sortie.
- **Apprentissage (Backpropagation)** : correction des poids en fonction de l'erreur.

---

## 3) Réseaux antagonistes génératifs (GANs)

### Composition

- **Générateur (G)** : crée des données artificielles à partir d'un vecteur de bruit  $z$ .
  - Exemple : image, son ou vidéo.
  - Architecture souvent basée sur **CNN** pour images.
  - Objectif : **tromper le Discriminateur**.
- **Discriminateur (D)** : classe les données comme réelles ou générées.
  - Entrée : données réelles ou générées.
  - Sortie : probabilité que la donnée soit réelle (1) ou fausse (0).
  - Objectif : **détecter les fausses données**.

## Vecteur de bruit (z)

- Point de départ du générateur.
- Composé de nombres aléatoires (distribution gaussienne ou uniforme).
- Sert de “graine” pour créer un échantillon.

## Fonction de perte (Loss Function)

- GAN = jeu **minimax** :
  - D maximise sa capacité à détecter les faux.
  - G minimise la probabilité d’être détecté.
- Formule :  
[  
$$\min_G \max_D V(D, G) = E_{x \sim P_{\text{data}}} [\log D(x)] + E_{z \sim P_z} [\log(1 - D(G(z)))]$$
  
]

## Processus d’entraînement

1. **Entraînement de D** :
  - Avec données réelles → doit dire “vrai”.
  - Avec données générées → doit dire “faux”.
2. **Entraînement de G** :
  - À partir de bruit aléatoire, G génère une donnée.
  - Objectif : convaincre D que la donnée est réelle.
3. **Boucle répétée** : G s’améliore, D devient plus exigeant.

---

Bien sûr ! Voici quelques **questions à choix multiple (QCM/QSM)** sur la partie **types de modèles d’IA générative et perceptrons/GANs**, avec les **réponses corrigées** :

---

### 1) Neurones artificiels et perceptrons

**Q1** : Quelle est la fonction principale d’un perceptron ?

- a) Classer les données uniquement
- b) Transformer des entrées en sortie via une somme pondérée et une fonction d’activation ✓
- c) Générer du bruit aléatoire
- d) Créer des images uniquement

**Q2** : Dans un perceptron, le **biais (b)** sert à :

- a) Calculer la somme pondérée
- b) Ajuster l’activation du neurone ✓

- c) Mesurer l'importance des entrées
- d) Créer le vecteur de bruit

**Q3 :** Parmi les fonctions d'activation suivantes, laquelle est non linéaire et très utilisée dans les réseaux neuronaux ?

- a) Sigmoid ☒
- b) Addition
- c) Multiplication
- d) Soustraction

**Q4 :** La **somme pondérée** d'un perceptron est :

- a)  $(z = \sum w_i + x_i)$
  - b)  $(z = \sum w_i * x_i + b)$  ☒
  - c)  $(z = \max(0, x_i))$
  - d)  $(z = x_i / w_i)$
- 

## 2) Architecture des réseaux neuronaux

**Q5 :** Dans un réseau de neurones, la **couche cachée** sert à :

- a) Recevoir les données brutes
- b) Transformer et extraire les caractéristiques ☒
- c) Produire la sortie finale
- d) Générer le bruit

**Q6 :** La **propagation avant (forward pass)** correspond à :

- a) Correction des poids
  - b) Calcul des sorties à partir des entrées ☒
  - c) Initialisation des poids
  - d) Ajout du biais
- 

## 3) Réseaux antagonistes génératifs (GANs)

**Q7 :** Dans un GAN, le **Générateur (G)** a pour rôle :

- a) Classer les données réelles ou fausses
- b) Créer des données artificielles ☒
- c) Calculer la fonction de perte
- d) Ajouter du bruit aux données réelles

**Q8 :** Le **Discriminateur (D)** :

- a) Génère des images réalistes
- b) Distingue les données réelles des données générées ☒

- c) Produit le vecteur de bruit
- d) Ajoute un biais aux entrées

**Q9 :** Le **vecteur de bruit (z)** dans un GAN est :

- a) Une donnée réelle
- b) Une entrée aléatoire pour générer des données ✓
- c) Une fonction d'activation
- d) La sortie finale du générateur

**Q10 :** Le processus d'entraînement d'un GAN consiste à :

- a) Former D seulement
- b) Former G seulement
- c) Former G et D en boucle, D essaye de détecter les faux et G essaye de les tromper ✓
- d) Ignorer les données réelles

---

Voici un **résumé clair et structuré** de la partie sur les **modèles de langage en IA générative** :

---

## ✦ Les modèles de langage (Language Models)

### 1) Définition

- Les modèles de langage sont **la base de l'IA générative appliquée au texte**.
- Objectif : **comprendre, prédire et générer du texte** en modélisant le langage humain.
- Principe fondamental : **prédire le mot suivant dans une séquence** selon ses probabilités d'occurrence.
- Importance : fondation des **chatbots, résumés automatiques, traduction, génération de contenu**.

---

### 2) Types de modèles de langage

#### *a) Modèles compacts*

- Taille : 1 million à 10 milliards de paramètres → **rapides et légers**.
- Spécialisation : entraînés pour une **tâche précise** (traduction, classification...).
- Consommation : **faible énergie**, adaptés à ordinateurs classiques, mobiles ou edge devices.



- Applications typiques :
  - Assistants vocaux sur smartphone ou objets connectés
  - Analyse de texte rapide (spam, classification)
  - Traduction spécialisée (ex : médical, juridique)
  - Applications sensibles à la vie privée (traitement local)
- Limites :
  - Compréhension limitée du **contexte long**
  - Moins performants sur **tâches complexes ou ouvertes**
  - Difficulté à **généraliser hors domaine**
  - Moins créatifs et polyvalents que les LLM

#### *b) Modèles larges (LLM – Large Language Models)*

- Taille : **milliards à centaines de milliards de paramètres.**
- Objectif : comprendre et générer du texte de manière **fluide et contextuelle.**
- Architecture : basée sur les **Transformers** avec **self-attention.**
- Données : entraînés sur **corpus massifs** (livres, articles, sites web, code).
- Applications :
  - Chatbots intelligents (ex : ChatGPT)
  - Résumés automatiques d'articles ou rapports
  - Traduction multilingue et compréhension du contexte global
  - Génération de contenu écrit : articles, emails, scripts, poèmes
  - Génération de code : GitHub Copilot, CodeWhisperer
  - Analyse de sentiments sur de grandes données textuelles
- Avantages :
  - **Compréhension avancée du langage**
  - **Polyvalence** pour texte, résumé, traduction, code
  - **Créativité et fluidité**
  - Adaptabilité via **fine-tuning**
  - Applications variées : entreprises, recherche, enseignement
- Limites :
  - **Hallucinations** : invente des informations plausibles mais incorrectes
  - **Biais** reproduits depuis les données d'entraînement
  - **Coût élevé** : GPU/TPU et forte consommation énergétique
  - **Problèmes éthiques et légaux** : droits d'auteur, protection des données

---

Parfait ! Voici un **QCM avec correction** sur la partie **modèles de langage en IA générative** :

---

## 1) Définition et principes

**Q1** : Quel est l'objectif principal d'un modèle de langage ?

a) Transformer des images

- b) Comprendre, prédire et générer du texte ✓
- c) Calculer des probabilités financières
- d) Analyser des signaux biologiques

**Q2 :** Le principe fondamental d'un modèle de langage est :

- a) Générer des nombres aléatoires
  - b) Prédire le mot suivant dans une séquence ✓
  - c) Classer des images
  - d) Créer des vidéos
- 

## 2) Modèles compacts

**Q3 :** Quelle caractéristique décrit le mieux un modèle compact ?

- a) Contient des milliards de paramètres
- b) Taille réduite, rapide et léger ✓
- c) Entraîné uniquement sur des données multilingues
- d) Toujours hébergé dans le cloud

**Q4 :** Les modèles compacts sont adaptés pour :

- a) Applications embarquées et mobiles ✓
- b) Génération de code complexe
- c) Traduction multilingue générale
- d) Création d'articles scientifiques de grande qualité

**Q5 :** Limite principale des modèles compacts :

- a) Trop coûteux à déployer
  - b) Compréhension limitée du contexte long ✓
  - c) Trop créatifs
  - d) Utilisent trop d'énergie
- 

## 3) Modèles larges (LLM)

**Q6 :** Quelle architecture est utilisée par les modèles larges comme GPT ou BERT ?

- a) Réseaux convolutifs (CNN)
- b) Transformers avec self-attention ✓
- c) Réseaux antagonistes (GAN)
- d) Perceptrons simples

**Q7 :** Parmi les applications suivantes, laquelle relève typiquement des modèles larges ?

- a) Détection rapide de spam
- b) Traduction multilingue, génération de texte ou code ✓

- c) Classification simple de documents
- d) Assistant vocal léger sur smartphone

**Q8 :** Limite d'un modèle large :

- a) Compréhension limitée du contexte
- b) Hallucinations et biais ✓
- c) Trop rapide à exécuter
- d) Peu d'applications

**Q9 :** Avantage d'un modèle large :

- a) Faible consommation d'énergie
- b) Polyvalence et créativité ✓
- c) Petit volume de données d'entraînement
- d) Toujours spécialisé dans un domaine

---

Voici un **résumé clair et structuré** de la partie sur **l'architecture d'un Transformer et le mécanisme d'attention** :

---

## ✦ Architecture d'un Transformer

### 1) Généralités

- Les **Transformers** sont à la base des LLM (Large Language Models).
- Ils surpassent les **RNN** et **LSTM** pour le traitement des séquences grâce à leur **traitement parallèle**.
- Applications : NLP (texte), vision par ordinateur, modèles avancés de génération.

### 2) Structure

- Composé de **deux parties principales** :
  1. **Encodeur (Encoder)** : comprend le contexte d'entrée.
  2. **Décodeur (Decoder)** : génère la sortie mot par mot.
- Les composants internes incluent :
  - **Input & Output Embeddings**
  - **Positional Encoding**
  - **Multi-Head Attention**
  - **Feed Forward**
  - **Add & Norm**
  - **Softmax & Linear** pour la sortie

---

# ✦ Mécanisme d'attention

## 1) Intuition

- Chaque mot dans une phrase peut avoir plusieurs références (ex. : “il” peut désigner “chat” ou “souris”).
- Le modèle apprend à **accorder plus d'attention aux mots importants** pour comprendre le contexte.

## 2) Représentation vectorielle

- Chaque mot  $\rightarrow$  vecteur (embedding).
- Trois projections linéaires : **Query (Q), Key (K), Value (V)**.

| Type      | Rôle                               | Analogie                               |
|-----------|------------------------------------|----------------------------------------|
| Query (Q) | Ce que le mot cherche à comprendre | “Je cherche à comprendre qui est ‘il’” |
| Key (K)   | Contenu proposé par le mot         | “Je suis ‘chat’, sujet de la phrase”   |
| Value (V) | Signification concrète du mot      | “Voici mon sens précis”                |

## 3) Calcul de l'attention

- Similarité entre **Q et K**  $\rightarrow$  produit scalaire.
- Normalisation  $\rightarrow$  distribution de probabilités (**softmax**).
- Pondération des **V** selon ces probabilités  $\rightarrow$  combinaison pondérée des mots.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Self-Attention (auto-attention)** : chaque mot regarde tous les autres mots d'entrée (encodeur).
- **Masked Self-Attention** : chaque mot regarde seulement les précédents (décodeur).
- **Encoder-Decoder Attention** : le décodeur regarde la sortie de l'encodeur pour générer le texte.

---

Parfait ! Voici un **QCM avec correction** sur l'**architecture des Transformers et le mécanisme d'attention** :

---

## 1) Généralités

**Q1 :** Les Transformers surpassent les RNN et LSTM grâce à :

- a) Leur capacité à traiter les séquences en parallèle ✓
- b) Leur faible consommation d'énergie
- c) Leur petite taille
- d) Leur utilisation exclusive en vision par ordinateur

**Q2 :** Les Transformers sont utilisés pour :

- a) Génération de texte, NLP, vision par ordinateur ✓
  - b) Calculs financiers uniquement
  - c) Base de données relationnelles
  - d) Applications mobiles légères
- 

## 2) Structure du Transformer

**Q3 :** Un Transformer est composé de :

- a) Un encodeur et un perceptron
- b) Un encodeur et un décodeur ✓
- c) Deux LSTM
- d) Un GAN et un CNN

**Q4 :** Dans le Transformer, le décodeur :

- a) Comprend le contexte d'entrée
  - b) Génère la sortie mot par mot ✓
  - c) Calcule les embeddings
  - d) Normalise les poids
- 

## 3) Mécanisme d'attention

**Q5 :** Quel est le rôle de la **Query (Q)** dans le mécanisme d'attention ?

- a) Contenu concret du mot
- b) Ce que le mot cherche à comprendre ✓
- c) Normaliser les scores
- d) Générer la sortie finale

**Q6 :** La **Key (K)** et la **Value (V)** représentent :

- a) K = ce que le mot cherche, V = pondération
- b) K = contenu du mot, V = signification concrète ✓
- c) K = sortie du décodeur, V = embedding d'entrée
- d) K = masque, V = attention multi-tête

**Q7 :** La formule de l'attention est :

- a)  $\text{Attention}(Q,K,V) = Q + K + V$
- b)  $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{dk}) V$  ✓
- c)  $\text{Attention}(Q,K,V) = Q * V / K$
- d)  $\text{Attention}(Q,K,V) = K^T V$

**Q8 :** La **Masked Self-Attention** est utilisée pour :

- a) Permettre à chaque mot du décodeur de regarder uniquement les mots précédents ✓
- b) Permettre à chaque mot de voir tous les autres mots
- c) Générer des embeddings
- d) Calculer les valeurs de sortie

**Q9 :** L'**Encoder-Decoder Attention** sert à :

- a) Masquer les entrées
- b) Faire regarder le décodeur aux sorties de l'encodeur ✓
- c) Normaliser les embeddings
- d) Créer des vecteurs aléatoires

---

---

### Les presentation

#### Exemples d'outils à présenter

| Type          | Outils possibles                                |
|---------------|-------------------------------------------------|
| <b>Texte</b>  | ChatGPT, Claude, Gemini, Mistral, LLaMA         |
| <b>Image</b>  | DALL·E 3, Midjourney, Stable Diffusion, Firefly |
| <b>Audio</b>  | ElevenLabs, MusicLM, AIVA                       |
| <b>Vidéo</b>  | RunwayML, Pika Labs, Sora                       |
| <b>Code</b>   | GitHub Copilot, Tabnine, AlphaCode              |
| <b>Autres</b> | Synthesia, ChatPDF, Hugging Face Spaces         |

---

Voici un **résumé clair et condensé** de la partie **Risques et limites de l'IA générative**, prêt pour lecture rapide et mémorisation :

---

## ✦ Risques et limites de l'IA générative

### 1) Limites générales

- L'IA générative crée du contenu nouveau : **texte, image, audio, vidéo.**

- Elle **n'a pas de conscience** et ne peut pas vérifier la vérité.
  - Peut produire :
    - **Erreurs factuelles**
    - **Biais ou stéréotypes**
    - **Contenus trompeurs**
- 

## 2) Risques techniques et éthiques

- **Hallucinations** : l'IA invente des faits non réels.
  - **Biais algorithmiques** : reproduction/amplification de préjugés sociaux.
  - **Fuites de données sensibles** : risque si des informations privées sont fournies.
  - **Manque de transparence** : décisions du modèle difficiles à expliquer.
  - **Dépendance excessive** : perte d'esprit critique ou de compétences humaines.
- 

## 3) Risques sociétaux

- **Propriété intellectuelle** : risque lié à l'utilisation de contenus existants.
  - **Désinformation / deepfakes** : création de faux contenus réalistes.
  - **Impact sur l'emploi** : automatisation de tâches créatives et rédactionnelles.
  - **Responsabilité légale** : ambiguïté sur qui est responsable en cas de préjudice.
- 

Voici un **QCM avec correction** sur la partie **Risques et limites de l'IA générative** :

---

**Q1** : L'IA générative peut produire du contenu :

- a) Seulement des textes
- b) Texte, image, audio et vidéo ✓
- c) Uniquement des images
- d) Aucun contenu

**Q2** : Qu'est-ce qu'une hallucination en IA générative ?

- a) Une image floue
- b) Une donnée inventée non réelle ✓
- c) Une erreur de calcul
- d) Une traduction automatique

**Q3** : Parmi ces options, laquelle est un risque technique ?

- a) Deepfakes
- b) Dépendance excessive ✓

- c) Impact sur l'emploi
- d) Propriété intellectuelle

**Q4 :** Les biais algorithmiques signifient :

- a) L'IA oublie les données anciennes
- b) L'IA reproduit ou amplifie des préjugés sociaux ✓
- c) L'IA crée des images floues
- d) L'IA ralentit le traitement

**Q5 :** Quel risque relève du domaine sociétal ?

- a) Hallucinations
- b) Fuites de données sensibles
- c) Automatisation de tâches créatives et rédactionnelles ✓
- d) Manque de transparence

**Q6 :** Le manque de transparence signifie :

- a) L'IA ne peut pas générer de contenu
- b) Les décisions du modèle sont difficiles à expliquer ✓
- c) L'IA fonctionne trop lentement
- d) L'IA produit uniquement des textes

**Q7 :** Pourquoi la responsabilité légale est un risque ?

- a) L'IA peut inventer des faits
- b) On ne sait pas qui est responsable en cas de préjudice ✓
- c) L'IA consomme beaucoup d'énergie
- d) L'IA nécessite un serveur puissant

---

## Chapitre 2

Voici un **résumé clair et structuré** de ton cours sur “**Maîtriser l’art du prompt**” :

---

## ✦ Chapitre 2 – Maîtriser l’art du prompt

### I. Introduction

- **Définition :** Un *prompt* est une instruction donnée à un modèle d'IA pour lui indiquer ce qu'on attend de lui.
- **Forme :** Phrase, paragraphe ou ensemble d'instructions.



- **Rôle fondamental** : Interface entre l'humain et la machine.
- **Impact** : La qualité du prompt influence directement la précision, la pertinence et la fiabilité de la réponse.

### Exemples :

- Prompt simple : "Explique le machine learning." → Réponse générale.
- Prompt détaillé : "Explique le machine learning à un étudiant de 2e année en data engineering, en mettant l'accent sur la logique mathématique derrière les réseaux de neurones." → Réponse ciblée et adaptée.

---

## II. Les 5 éléments essentiels d'un prompt (RCOCF)

1. **Rôle – “Qui parle ?”**
  - Donner au modèle une identité ou expertise.
  - Influence le style et le ton.
  - **Exemples** :
    - "Tu es un professeur qui enseigne Python à des débutants."
    - "Tu es un consultant en cybersécurité."
2. **Contexte – “Dans quelle situation ?”**
  - Fournir le cadre de la demande.
  - Le modèle doit connaître le public, le domaine et les contraintes.
  - **Exemples** :
    - "Tu t'adresses à des étudiants de 2e année en data engineering."
    - "Le texte provient d'un rapport interne."
3. **Objectif – “Que doit faire le modèle ?”**
  - Définir l'action précise : expliquer, résumer, analyser, générer, etc.
  - **Exemples** :
    - "Explique le fonctionnement d'un pipeline de données."
    - "Compare CNN et Transformer."
4. **Contraintes – “Sous quelles conditions ?”**
  - Définir la longueur, le ton, le niveau de détail ou les exclusions.
  - **Exemples** :
    - "Fais une explication courte (moins de 150 mots)."
    - "Utilise un langage simple et pédagogique."
5. **Format – “Sous quelle forme livrer la réponse ?”**
  - Définir la présentation : texte, liste, tableau, code, JSON, Markdown...
  - **Exemples** :
    - "Présente la réponse en tableau."
    - "Fournis un script Python complet avec commentaires."

### Exemple complet de prompt structuré (RCOCF) :

```
[Rôle] Tu es un enseignant en data engineering.
[Contexte] Tu dois expliquer un concept à des étudiants de 2e année.
[Objectif] Explique le principe d'un pipeline de données.
```

[Contraintes] Utilise un langage simple, sans équations.  
[Format] Présente la réponse en 3 étapes numérotées.

---

Voici un **QCM avec correction** basé sur ton cours “**Maîtriser l’art du prompt**” :

---

## QCM – Maîtriser l’art du prompt

**Question 1 :** Qu’est-ce qu’un *prompt* en IA générative ?

- A) Une base de données pour entraîner un modèle
- B) Une instruction donnée à un modèle d’IA pour indiquer ce qu’on attend de lui
- C) Un type de réseau de neurones
- D) Une fonction mathématique pour calculer la probabilité

✓ **Réponse :** B

**Explication :** Un prompt sert à guider le modèle pour produire la réponse attendue.

---

**Question 2 :** Parmi les éléments suivants, lequel fait **partie des 5 éléments essentiels d’un prompt** ?

- A) Rôle
- B) Algorithme
- C) Biais
- D) Optimisation

✓ **Réponse :** A

**Explication :** Les 5 éléments sont : Rôle, Contexte, Objectif, Contraintes, Format (RCOCF).

---

**Question 3 :** Que représente le **Contexte** dans un prompt ?

- A) Le style de réponse attendu
- B) La situation dans laquelle la demande est faite et les informations nécessaires pour la comprendre
- C) Le format de sortie (tableau, JSON...)
- D) L’identité du modèle

✓ **Réponse :** B

**Explication :** Le contexte précise le public, le domaine et les conditions pour que la réponse soit adaptée.

---

**Question 4 :** Quel élément permet de définir la **forme de la réponse** (texte, tableau, JSON...) ?

- A) Rôle
- B) Contexte
- C) Format
- D) Objectif

✓ **Réponse :** C

**Explication :** Le format indique comment la réponse doit être structurée pour être exploitable.

---

**Question 5 :** Pourquoi un **bon prompt** est-il important ?

- A) Il réduit le temps de calcul du modèle
- B) Il améliore la qualité, la précision et la pertinence de la réponse
- C) Il entraîne automatiquement le modèle
- D) Il transforme les données en vecteurs

✓ **Réponse :** B

**Explication :** Un prompt clair et structuré guide l'IA et évite les réponses vagues ou incorrectes.

---

**Question 6 :** Parmi les exemples suivants, lequel est un **bon prompt** ?

- A) "Explique-moi le deep learning."
- B) "Donne-moi un code Python."
- C) "Explique le deep learning à un étudiant en 2e année d'ingénierie des données, en mettant l'accent sur la logique mathématique derrière les réseaux de neurones."
- D) "Parle-moi de l'IA."

✓ **Réponse :** C

**Explication :** Ce prompt est clair, structuré, contextualisé et cible le public.

---

**Question 7 :** Dans le modèle **RCOCF**, que signifie "C" ?

- A) Créativité
- B) Contexte
- C) Calcul
- D) Contraintes

✓ **Réponse :** B

**Explication :** RCOCF = Rôle – Contexte – Objectif – Contraintes – Format

---

Voici un **résumé clair et structuré** de la partie “Cadres structurés de prompts (Prompt Frameworks)” :

---

## ✦ III. Cadres structurés de prompts (Prompt Frameworks)

### 1. Définition

Un *framework de prompt* est un **cadre structuré** qui aide à formuler des requêtes efficaces pour interagir avec un modèle d’IA générative.

- Composé des éléments : **Rôle – Contexte – Objectif – Contraintes – Format** (RCOCF).
  - Permet de **tester un même prompt sur différents modèles** (GPT, Claude, Gemini) et comparer les performances.
- 

### 2. Principaux frameworks

| Framework     | Utilité                                     | Structure / Exemple                                                                                                                                                                                                                                                             |
|---------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RACE</b>   | Expert ciblé, réponses spécialisées         | <b>Role</b> : Tu es un conseiller financier. <b>Action</b> : Crée une stratégie d’épargne. <b>Context</b> : Public 25–30 ans avec dettes étudiantes. <b>Expectation</b> : 6 étapes claires, ton motivant, format numéroté.                                                      |
| <b>TAG</b>    | Simple et rapide                            | <b>Task</b> : Résume un article sur l’énergie verte. <b>Action</b> : Mets en avant innovations clés. <b>Goal</b> : Résumé clair et concis pour décideur.                                                                                                                        |
| <b>TRACE</b>  | Exemples concrets pour guider la génération | <b>Task</b> : Créer 2 descriptions de produits. <b>Request</b> : Mettre en avant un avantage. <b>Action</b> : Inclure un appel à l’action. <b>Context</b> : Public jeunes adultes soucieux de l’environnement. <b>Example</b> : “Hydratez-vous tout en protégeant la planète !” |
| <b>CARE</b>   | Centré sur contexte et résultat             | <b>Context</b> : Pré-lancement d’une app. <b>Action</b> : Rédige un email de recrutement. <b>Result</b> : 200 mots, ton engageant. <b>Example</b> : “Soyez les premiers à découvrir LingoLeap !”                                                                                |
| <b>BAB</b>    | Storytelling convaincant                    | <b>Before</b> : Trop de réunions improductives. <b>After</b> : Journées concentrées et productives. <b>Bridge</b> : 5 solutions concrètes.                                                                                                                                      |
| <b>C.O.T.</b> | Raisonnement étape par étape                | <b>Chain Of Thought</b> : Raisonne étape par étape avant de répondre à une question scientifique ou logique.                                                                                                                                                                    |

---

### 3. Atelier pratique : Devenir concepteur de prompts

- Objectif : créer un prompt pour **soutenir la santé mentale, motivation ou bien-être**.
  - Étapes :
    1. Former une équipe (3–5 participants).
    2. Choisir un cadre (RACE, TAG, TRACE, CARE, BAB, C.O.T.).
    3. Construire le prompt complet à partir du cadre choisi.
    4. Tester le prompt avec une IA (ex. ChatGPT).
    5. Améliorer le prompt selon les résultats.
- 

Voici un **QCM clair avec correction** basé sur les cadres structurés de prompts (Prompt Frameworks) :

---

#### QCM – Cadres structurés de prompts

1 ☐ **Qu'est-ce qu'un framework de prompt ?**

- A) Un modèle d'IA qui génère du texte automatiquement
- B) Un cadre structuré pour formuler des requêtes efficaces à l'IA
- C) Une base de données pour stocker des prompts
- D) Une bibliothèque de codes Python pour NLP

**Réponse : B** ✓

*Explication : Un framework de prompt sert à organiser et structurer les instructions données à un modèle d'IA.*

---

2 ☐ **Le framework RACE est principalement utilisé pour :**

- A) Des résumés rapides d'articles
- B) Faire des analyses statistiques simples
- C) Obtenir des réponses spécialisées et expertes
- D) Créer des images générées par IA

**Réponse : C** ✓

*Explication : RACE transforme le modèle en expert ciblé (ex : conseiller financier, chercheur, spécialiste marketing).*

---

3 ☐ **Dans le framework TAG, que représente le "G" ?**

- A) Goal → Résultat attendu
- B) Generate → Générer du contenu

- C) Guide → Guide de style
- D) Group → Public cible

**Réponse : A** ✓

*Explication : TAG signifie Task, Action, Goal. "Goal" indique le résultat attendu.*

---

**4** ☐ **Quel framework est le plus adapté pour guider la génération avec un exemple concret ?**

- A) RACE
- B) TRACE
- C) CARE
- D) BAB

**Réponse : B** ✓

*Explication : TRACE inclut un exemple de sortie attendu pour orienter la génération.*

---

**5** ☐ **Le BAB framework est utilisé pour :**

- A) Storytelling et communication persuasive
- B) Analyse scientifique étape par étape
- C) Résumés rapides et synthèses
- D) Créer des prompts pour traductions

**Réponse : A** ✓

*Explication : BAB structure le message autour d'un problème (Before), d'une solution (Bridge) et d'un futur désirable (After).*

---

**6** ☐ **Le C.O.T. framework aide l'IA à :**

- A) Générer du texte créatif pour des histoires
- B) Raisonner étape par étape avant de répondre
- C) Traduire automatiquement des phrases
- D) Créer des tableaux et listes de données

**Réponse : B** ✓

*Explication : C.O.T. = Chain Of Thought, idéal pour raisonnement logique, scientifique ou mathématique.*

---

**7** ☐ **Le framework CARE est surtout utilisé pour :**

- A) Des analyses scientifiques complexes

- B) Centrer la réponse sur un contexte et un résultat attendu
- C) Créer des images réalistes avec IA
- D) Résumer des textes longs pour décideurs

**Réponse : B** ✓

*Explication : CARE fournit le contexte, l'action, le résultat attendu et éventuellement un exemple de ton ou format.*

---

## Chapitre3

### ✓ Résumé global du Chapitre 3 : Les outils pour le développement d'applications d'IA générative

Une application d'IA générative n'est pas seulement un modèle comme GPT ou LLaMA. C'est **un système complet composé de plusieurs couches**, chacune jouant un rôle essentiel. Voici les 5 couches principales :

---

#### ☐ I. Fondations et architecture d'une application d'IA générative

Une application GenAI =

☞ **Modèle + Framework applicatif + Orchestration + Mémoire + Interface**

---

#### 1 ☐ Couche 1 : Le Modèle Génératif (Foundation Model)

C'est le **cerveau de l'application**.

**Rôle :**

- Comprendre les prompts de l'utilisateur.
- Générer du texte, des images, etc.

**Exemples :**

- **LLM texte** : GPT-4, LLaMA 3, Mistral.
- **Images** : Stable Diffusion, DALL·E 3, Midjourney.

**Accès :**

- **APIs commerciales** : OpenAI, Anthropic, Mistral.
- **Open-source** : Hugging Face (Transformers, Diffusers).

---

## 2 Couche 2 : Le Framework Applicatif

C'est la couche qui **organise la logique de l'application** autour du modèle.

**Ce qu'elle permet :**

- Structurer les prompts (prompt engineering).
- Gérer la mémoire conversationnelle.
- Connecter les LLMs avec des documents et bases de données.
- Enchaîner plusieurs étapes (workflow logique).

**Outils principaux :**

| Framework   | Fonction                               |
|-------------|----------------------------------------|
| LangChain   | Construire des chaînes IA, agents, RAG |
| LlamaIndex  | Connecter documents → LLM              |
| PromptLayer | Versionner et suivre les prompts       |

---

## 3 Couche 3 : L'Orchestration et l'Automatisation



Elle sert à **coordonner toutes les étapes** d'une application IA.

**Rôle :**

- Exécuter automatiquement des pipelines IA.
- Connecter modèles, APIs, bases vectorielles.
- Planifier des tâches automatiques.

**Outils :**

| Outil          | Description                                |
|----------------|--------------------------------------------|
| n8n            | Outil low-code pour créer des workflows IA |
| Apache Airflow | Orchestration de pipelines complexes       |
| Prefect        | Alternative moderne à Airflow              |

---

## 4 Couche 4 : Le Stockage et la Mémoire (Vector Databases)

Les modèles ne retiennent pas tout → cette couche apporte **mémoire et contexte**.

**Rôle :**

- Stocker des embeddings (représentations vectorielles).
- Chercher les documents les plus pertinents.
- Fournir un contexte au modèle (RAG).

**Outils :**

| Outil             | Fonction                       |
|-------------------|--------------------------------|
| FAISS             | Recherche vectorielle (Meta)   |
| Pinecone          | Base vectorielle hébergée      |
| Chroma / Weaviate | Solutions open source pour RAG |

---

## 5 Couche 5 : Interface et Déploiement

C'est ce qui rend l'application accessible aux utilisateurs.

**Types d'interface :**

| Type                    | Outils            | Usage                       |
|-------------------------|-------------------|-----------------------------|
| <b>Interface Web</b>    | Gradio, Streamlit | Application visuelle simple |
| <b>API Backend</b>      | FastAPI, Flask    | Donner accès par API        |
| <b>Conteneurisation</b> | Docker            | Déploiement et isolation    |

---

## □ Exemple complet d'un stack GenAI

| Couche        | Outil                   |
|---------------|-------------------------|
| Modèle        | GPT-4 ou Claude 3       |
| Framework     | LangChain ou LlamaIndex |
| Mémoire       | Chroma ou Pinecone      |
| Interface     | Gradio / FastAPI        |
| Orchestration | n8n                     |

---

## □ Résumé ultra-court (pour examen) :

Une application d'IA générative est construite en 5 couches :

1. **Modèle génératif** → génère le langage (GPT, LLaMA...).
  2. **Framework applicatif** → organise les prompts, la logique, les données (LangChain).
  3. **Orchestration** → automatise et coordonne (n8n, Airflow).
  4. **Mémoire vectorielle** → store embeddings et fait recherche sémantique (FAISS, Pinecone).
  5. **Interface / Déploiement** → interface, API, Docker (Gradio, FastAPI).
- 

## QCM (20 questions) – Chapitre 3 : Outils pour le développement d'applications d'IA Générative

---

1. Une application d'IA générative est composée de :

- A. Un seul modèle LLM
- B. Plusieurs couches : modèle, framework, orchestration, mémoire, interface
- C. Une base de données seulement
- D. Une interface graphique seulement

✓ **Correction : B**

---

## **2. Le “cerveau” d’une application GenAI est :**

- A. Le framework LangChain
- B. Le modèle génératif (Foundation Model)
- C. L’interface utilisateur
- D. L’orchestrateur n8n

✓ **Correction : B**

---

## **3. Quel est un exemple de modèle génératif texte ?**

- A. Stable Diffusion
- B. GPT-4
- C. Docker
- D. Gradio

✓ **Correction : B**

---

## **4. Quel outil est utilisé pour générer des images ?**

- A. LLaMA 3
- B. FastAPI
- C. Midjourney
- D. LangChain

✓ **Correction : C**

---

## **5. Hugging Face Transformers permet :**

- A. Créer des interfaces web
- B. Accéder à des modèles open-source
- C. Stocker des embeddings
- D. Automatiser des workflows

✓ **Correction : B**

---

## **6. Le rôle d'un framework applicatif est de :**

- A. Planifier des tâches
- B. Fournir une interface web
- C. Organiser la logique autour du modèle
- D. Exécuter Docker

✓ **Correction : C**

---

## **7. Quel framework permet de faire des chaînes logiques, RAG ou agents ?**

- A. LangChain
- B. Midjourney
- C. Docker
- D. FAISS

✓ **Correction : A**

---

## **8. LlamaIndex sert principalement à :**

- A. Déployer des APIs
- B. Connecter des documents aux LLMs
- C. Générer des images
- D. Faire du monitoring

✓ **Correction : B**

---

## 9. Quel outil permet de versionner et suivre les prompts ?

- A. PromptLayer
- B. Airflow
- C. Gradio
- D. Stable Diffusion

✓ Correction : A

---

## 10. L'orchestration sert à :

- A. Stocker des embeddings
- B. Connecter et automatiser les étapes d'une application IA
- C. Déployer une interface graphique
- D. Traduire les prompts

✓ Correction : B

---

## 11. Quel outil est un orchestrateur low-code ?

- A. n8n
- B. Pinecone
- C. Streamlit
- D. GPT-4

✓ Correction : A

---

## 12. Apache Airflow est utilisé pour :

- A. Générer des images
- B. Gérer des pipelines de données complexes
- C. Créer des prompts
- D. Construire des interfaces graphiques

✓ Correction : B

---

### 13. Une base vectorielle sert à :

- A. Compiler des modèles
- B. Stocker des pages HTML
- C. Rechercher du contenu pertinent via embeddings
- D. Générer des images

✓ Correction : C

---

### 14. Quel outil est une base vectorielle open source ?

- A. FAISS
- B. Docker
- C. LangChain
- D. Streamlit

✓ Correction : A

---

### 15. Pinecone est :

- A. Un modèle LLM
- B. Un orchestrateur
- C. Une base vectorielle hébergée
- D. Un modèle de diffusion

✓ Correction : C

---

### 16. L'interface web d'une application peut être créée avec :

- A. Streamlit ou Gradio
- B. Airflow
- C. LLaMA 3
- D. Pinecone

✓ Correction : A

---

## 17. FastAPI sert à :

- A. Créer des APIs backend
- B. Générer des vecteurs
- C. Automatiser des pipelines
- D. Entraîner des modèles

✓ Correction : A

---

## 18. Docker est utilisé pour :

- A. Déployer et isoler une application
- B. Créer des prompts
- C. Faire du RAG
- D. Automatiser des workflows

✓ Correction : A

---

## 19. La couche mémoire utilise principalement :

- A. Des images
- B. Des embeddings
- C. Du texte brut
- D. Des vidéos

✓ Correction : B

---

## 20. Un “stack” complet d’application GenAI contient :

- A. Only GPT-4
- B. Modèle + Framework + Mémoire + Interface + Orchestration
- C. Un GPU seulement
- D. Une base SQL

✓ Correction : B

---

Voici **un résumé clair, simple et MAXI-COURT** du cours **II. Plateformes de modèles et Bibliothèques** — parfait pour réviser rapidement.

---

# **Résumé Ultra-Simplifié : Plateformes de modèles & Bibliothèques**

## ◆ 1. Rôle des plateformes de modèles

Les plateformes de modèles = **les app stores de l'IA**.  
Elles servent à :

1. **Centraliser** les modèles pré-entraînés.
2. **Standardiser** : API, formats, licences.
3. **Faciliter la recherche**, les tests et le partage.
4. **Encourager la collaboration** (développeurs, ingénieurs, chercheurs...).

---

## ◆ 2. Types de plateformes

| Type                               | Exemples                             | Avantages                             | Limites                                 |
|------------------------------------|--------------------------------------|---------------------------------------|-----------------------------------------|
| <b>Open source</b>                 | Hugging Face, GitHub, TensorFlow Hub | Transparent, modifiable, flexible     | Besoin de ressources locales            |
| <b>Propriétaires / API fermées</b> | OpenAI, Anthropic, Mistral API       | Simple, puissant, maintenance assurée | Coût, dépendance, pas d'accès aux poids |

---

## ◆ 3. Hugging Face : plateforme open source de référence

- Créée en 2016 (au début un petit chatbot).
- Aujourd'hui : **GitHub de l'IA**.
- Sert à :
  - diffuser les modèles (LLM, vision, audio, multimodalité),
  - centraliser jeux de données,
  - héberger des applications IA (Spaces),
  - démocratiser l'IA.

**Ses 3 grands hubs :**

- **Model Hub** → +1 million de modèles.
- **Datasets Hub** → milliers de datasets publics.
- **Spaces Hub** → applications IA prêtes à tester (Gradio, Streamlit).



---

#### ◆ 4. Principales bibliothèques Hugging Face

| Bibliothèque        | Fonction                                   |
|---------------------|--------------------------------------------|
| <b>transformers</b> | Utiliser modèles NLP/LLM                   |
| <b>datasets</b>     | Télécharger, préparer les datasets         |
| <b>diffusers</b>    | Génération d'images, audio, vidéo          |
| <b>gradio</b>       | Créer interfaces IA                        |
| <b>accelerate</b>   | Entraînement optimisé, GPU, multi-machines |

---

#### ◆ 5. TP : Premiers pas sur Hugging Face

1. Tester **3 modèles** (ex : BERT, GPT-2).
  2. Explorer **1 dataset** (ex : IMDB, AG News).
  3. Tester **2 Spaces** (demo IA).
  4. Rédiger un petit compte-rendu : ce que tu as appris, ce qui t'a surpris.
- 

#### ◆ 6. Bibliothèque Transformers

La bibliothèque principale de Hugging Face.  
Elle permet d'utiliser facilement :

- modèles NLP,
- modèles vision,
- modèles audio (ASR, TTS),
- modèles multimodaux,
- GPT, BERT, T5, LLaMA, Mistral, etc.

#### Avantages :

- open source
- API simple
- compatible PyTorch / TensorFlow
- pipelines haut niveau (faciles à utiliser)

#### Cas d'usage :

- résumé, génération de texte
- traduction
- classification de texte / émotions
- agents conversationnels
- classification/segmentation d'images
- reconnaissance vocale

- VQA (questions visuelles)

---

## ◆ 7. Structure générale de Transformers

- **Model** : architecture (GPT, BERT, ViT...).
- **Pipeline** : exécute une tâche complète (texte → résultat).
- **Trainer** : outil pour l'entraînement.

---

## ◆ 8. Tokens d'accès Hugging Face

Clés secrètes pour accéder :

- aux modèles privés,
- aux datasets privés,
- aux APIs,
- pour publier modèles / datasets / Spaces.

**Types de permissions :**

- **READ** → télécharger / API
- **WRITE** → publier
- **ADMIN** → gérer permissions

---

## 🔑 Résumé en 6 mots

Plateformes regroupent – HF domine – Transformers facilite.

---

Voici un **QCM d'examen + correction** basé sur ton cours **Plateformes de modèles & Bibliothèques (Hugging Face)**.

Format parfait pour réviser 100.

---

## 📌 QCM – Plateformes de modèles & Bibliothèques

(Correction à la fin)

---

Q1. Les plateformes de modèles sont comparables à :

- A. Des réseaux sociaux
- B. Des app stores de l'IA
- C. Des systèmes d'exploitation
- D. Des navigateurs web

---

Q2. Quel est l'un des rôles principaux des plateformes de modèles ?

- A. Augmenter la vitesse du processeur
- B. Centraliser et accéder à des modèles pré-entraînés
- C. Créer des cartes graphiques
- D. Remplacer les data centers

---

Q3. Quel type de plateforme donne accès au code source et aux poids du modèle ?

- A. API fermée
- B. Plateforme propriétaire
- C. Open source
- D. Cloud Gaming

---

Q4. Quel est un exemple de plateforme propriétaire ?

- A. Hugging Face
- B. GitHub
- C. OpenAI
- D. TensorFlow Hub

---

Q5. Hugging Face a été créé à l'origine comme :

- A. Une base de données
  - B. Une application chatbot
  - C. Un moteur de recherche
  - D. Un système d'exploitation
-

Q6. Quel hub de Hugging Face contient plus d'un million de modèles ?

- A. Space Hub
  - B. Model Hub
  - C. Transformers Hub
  - D. Dataset Hub
- 

Q7. Quelle bibliothèque est utilisée pour générer des images, audio ou vidéo ?

- A. transformers
  - B. datasets
  - C. diffusers
  - D. accelerate
- 

Q8. Le pipeline dans Transformers sert à :

- A. Optimiser le GPU
  - B. Créer une interface web
  - C. Exécuter une tâche complète facilement
  - D. Télécharger des datasets
- 

Q9. Quel type de permission permet de publier un modèle sur Hugging Face ?

- A. READ
  - B. WRITE
  - C. DOWNLOAD
  - D. ADMIN uniquement
- 

Q10. Les tokens d'accès servent à :

- A. Démarrer l'ordinateur
  - B. Acheter des modèles
  - C. S'authentifier et accéder à des ressources HF
  - D. Augmenter la RAM
- 
- 

 **CORRECTION**

**Q1 → B**

Les plateformes de modèles = "app stores" de l'IA.

**Q2 → B**

Elles servent à centraliser les modèles pré-entraînés.

**Q3 → C**

Open source = accès au code, modification, transparence.

**Q4 → C**

OpenAI = API fermée (pas accès aux poids).

**Q5 → B**

Hugging Face était au début une petite app chatbot.

**Q6 → B**

Model Hub contient +1 million de modèles.

**Q7 → C**

diffusers = génération d'images, audio, vidéo.

**Q8 → C**

Le pipeline relie tokenizer + modèle pour exécuter une tâche complète.

**Q9 → B**

WRITE = publier un modèle / dataset.

**Q10 → C**

Token = authentification + accès aux ressources HF.